

document	title	author	date	target	audience
P2616R1	Making std::atomic notification/wait operations usable in more situations	Lewis Baker lewissbaker@gmail.com	2022- 11-09	C++26	SG1

- [Abstract](#)
- [Motivation](#)
- [Option 1 - Allow namespace-scope function `std::atomic\_notify\_one/all` to accept pointer to potentially-destroyed object](#)
- [Option 2 - Add fused store-and-notify methods for each atomic operation](#)
 - [Other implementations considered](#)
- [Conclusion](#)
- [References](#)

Revision History

R1

- Added Option 3 - `atomic_notify_token` approach.
- Incorporate feedback on limitations of option 2.

R0

Proposed two options.

- Option 1 - allow `std::atomic_notify_one/all()` on potentially destroyed pointer
- Option 2 - add fused modify + notify overloads, `std::memory_notification` enum

Abstract

C++20 added support to `std::atomic` for `notify_one()`, `notify_all()` and `wait()` operations.

However, the separation of the notify operation from atomic-store operations makes it difficult and/or inefficient to use these operations for building certain kinds of synchronisation primitives (e.g. `std::counting_semaphore`) in a portable way.

In cases where a thread signalling a synchronisation primitive might release another thread waiting for that signal which then subsequently destroys the synchronisation primitive (for example in implementation of the `sync_wait()` primitive from P2300), a naïve usage of `std::atomic` that stores a value and then calls `std::atomic::notify_one()` might end up calling `notify_one()` on a destroyed object if the waiting thread sees the store and immediately completes without waiting to be notified and destroys the atomic object before the signalling thread completes the call to `notify_one()`.

This paper proposes three potential directions for solving this, for consideration by the concurrency sub-group.

1. Change the semantics of namespace-scope `std::atomic_notify_one()` and `std::atomic_notify_all()` to allow passing a pointer to a `std::atomic` object whose lifetime may have ended.
2. For each operation that can result in a store to the atomic variable, add a new overload that takes a `std::memory_notification` enum value and that fuses the store operation with a corresponding notify operation such that the operations are (as if) performed atomically.
3. Provide the ability to obtain an `atomic_notify_token` from the atomic object which can be used to safely notify threads waiting on the atomic object even if the atomic object is potentially destroyed.

The first option is arguably a simpler change to the specification and is possibly the preferable approach if concerns about pointer providence and the potential for undefined-behaviour passing pointers to destroyed objects can be overcome with core language changes proposed by the "pointer zap" papers (P1726 and P2188).

The second and third options do not have the same requirements for core language changes and thus can provide a solution independent of core-language changes.

The second option is a much larger interface change to `std::atomic`, involving adding about 20 new overloads to the `std::atomic` and `std::atomic_ref` classes and additional overloads of the namespace-scope atomic functions. However, it also has some limitations in terms of ability to support safe conditional notification based on the previous value of a RMW operation.

The third option is a much smaller library change - adding a single new method to `std::atomic` and `std::atomic_ref` and also adding a new type `std::atomic_notify_token`. This option does not have the same limitations on usage as option 2.

I am seeking guidance from the Concurrency sub-group on the preferred direction for addressing the issue.

Motivation

Consider the following simplified case:

```
int main() {
    thread_pool tp;

    {
        std::atomic<int> x{0}; // #1
        tp.execute([&] {      // #2
            x.store(1);        // #3
            x.notify_one();     // #4
        });
        x.wait(0);             // #5
        assert(x.load() == 1);
    }                          // #6
}
```

Let T1 be the thread-pool thread that executes the lambda and T2 be the thread executing `main()`.

We might end up with a situation where the following sequence of events is executed:

```
T2: constructs the atomic `x` and initialises to 0 (#1)
T2: enqueues the task to execute on the T1 (#2)
T1: dequeues the task and invokes the lambda
T1: stores the value 1 to the `x` (#3)
T2: executes `x.wait(0)`, sees value is now 1 and returns immediately (#5)
T2: destroys the object `x` (#6)
T1: executes `x.notify_one()` on a now destroyed object `x` (#4)
```

To work around this potential for operating on a dangling reference, we can use two separate atomic variables - `wait()` on one and then spin-wait on the other.

```

int main() {
    thread_pool tp;

    {
        std::atomic<int> x{0};
        std::atomic_flag f{true};
        tp.execute([&] {
            x.store(1);
            x.notify_one();
            f.clear();
        });
        x.wait(0);           // efficient-wait
        while (f.test());    // spin-wait
        assert(x.load() == 1);
    }
}

```

This has a couple of downsides, however:

- It needs to perform two atomic operations instead of one
- The spin-wait can still result in occasional long busy-wait times, wasting CPU resources e.g. if the signalling thread is context-switched out by the OS after the call to `x.notify_one()` but before the call to `f.clear()`, the main thread will busy-wait spin until the signalling thread is rescheduled.

For this particular situation, we could instead use a `std::binary_semaphore`, as its `acquire()` and `release()` operations are defined as being atomic, and thus they do not have the same lifetime issues as the first `std::atomic` implementation.

```

int main() {
    thread_pool tp;

    {
        std::binary_semaphore sem{0};
        tp.execute([&] {
            sem.release();
        });
        sem.acquire();
    }
}

```

However, if we then ask the question "Can we implement `std::counting_semaphore` using `std::atomic`?", we quickly run into the same lifetime questions regarding use of `notify_one/all()` methods.

Indeed, if we look at some of the standard library implementations of

`std::counting_semaphore` we see that they do actually follow the same pattern as above for the `release()` operation - an atomic store followed by a call to either `notify_all()` or `notify_one()` on the atomic object.

- See [libc++ `counting\_semaphore::release\(\)` implementation](#).
- See [libstdc++ `counting\_semaphore::release\(\)` implementation](#)
- See [msvc `counting\_semaphore::release\(\)` implementation](#)

So why don't they run into the same lifetime issues?

The reason is that in all major standard library implementations of

`std::atomic::notify_all()` and `std::atomic::notify_one()` depend only on the address of the atomic object, but do not actually access any data-members of the atomic object.

The underlying implementations of these notifying functions typically boil down to a call to:

- `WakeByAddressSingle()` / `WakeByAddressAll()` on Windows platforms
- `futex()` with `futex_op` set to `FUTEX_WAKE` on Linux platforms
- `__unlock_wake()` on Mac OS platforms
- Use of one of a collection of statically-allocated condition-variables, with the particular one chosen based on the bit-representation of the provided address.

All of these depend only on the bit-representation of address of the atomic variable, which makes calling `count.notify_all()` work, even though the `count` object may have since been destroyed, as it doesn't try to dereference the pointer.

Standard library implementations have extra knowledge about the semantics of the `std::atomic::notify_one/all()` methods which they can leverage to allow safely implementing `std::counting_semaphore` in terms of `std::atomic` operations. However, users of the standard library which want to be portable cannot make these same assumptions - the standard library does not currently require that all implementations have this same behaviour, and so users must resort to other, more portable means.

Instead of requiring users to work-around these limitations of the interface, we should provide some standard/portable way for users to safely modify an atomic value and notify waiting threads of the change.

Option 1 - Allow namespace-scope function

`std::atomic_notify_one/all` to accept pointer to potentially-destroyed object

The first option for solving this proposal is to change the semantics of `std::atomic_notify_all` and `std::atomic_notify_one` functions to allow accepting a pointer to a potentially-destroyed object. i.e. have them guarantee that they will not try to access the object pointed-to by the provided pointer.

With this change, the above example could be written as follows:

```
int main() {
    thread_pool tp;

    {
        std::atomic<int> x{0};
        tp.execute([&] {
            auto* px = &x;
            x.store(1);
            std::atomic_notify_one(px);
        });
        x.wait(0);
        assert(x.load() == 1);
    }
}
```

The example here takes the address of `x` before executing the `x.store(1)` operation as the object `x` may not be valid after the `x.store(1)` statement is executed.

However, this is arguably not significantly different to just taking the address of `x` after the store. Indeed, the validity of this code depends on whether or not it is valid to use the pointer `px` at a point in the program where the pointed-to object is potentially destroyed.

There has been much debate about the semantics of using pointers-to-destroyed objects in the papers:

- [P1726R5](#) "Pointer lifetime-end zap and provenance, too" (Paul McKenney, Maged Michael, et. al.)
- [P2188R1](#) "Zap the Zap: Pointers are sometimes just bags of bits"

The storage duration section `[basic.stc.general]`, paragraph 4 reads:

When the end of the duration of a region of storage is reached, the values of all pointers representing the address of any part of that region of storage become invalid pointer values (`[basic.compound]`). Indirection through an invalid pointer value and passing an invalid pointer value to a deallocation function have undefined behavior. Any other use of an invalid pointer value has implementation-defined behavior. `[26]` `[26]` Some implementations might define that copying an invalid pointer value causes a system-generated runtime fault.

Thus, depending on the outcomes of the discussions of these papers, this proposed solution may or may not be valid. According to the current wording of `[basic.stc.general]`, passing the pointer to the `std::atomic_notify_one()` function has implementation-defined behaviour.

We would need wording changes to `[basic.stc.general]` that makes it valid to pass the address of a potentially destroyed object of type `std::atomic<T>` to the `std::atomic_notify_one/all` function. This may be too big or too wide-ranging a change to add just for enabling `std::atomic<T>::notify_one/all` operations.

Option 2 - Add fused store-and-notify methods for each atomic operation

The alternative to the above approach is to instead define new methods on a `std::atomic` object that atomically perform the store + notify steps such that there is no potential for invoking undefined/implementation-defined behaviour.

The implementation itself may still be taking the address of the object and performing separate store + notify steps. However, as whether or not this is valid is implementation defined, presumably the implementation can do this in such a way that it does not invoke the undefined behaviour that would be involved if user-code were to attempt the same thing.

This option proposes adding the enum `memory_notification` : and the following overloads to the `std::atomic` class:

```
namespace std {
    enum class memory_notification : unspecified {
        notify_none = unspecified,
        notify_one = unspecified,
        notify_all = unspecified
    };
    inline constexpr auto memory_notify_none = memory_notification::notify_none;
    inline constexpr auto memory_notify_one = memory_notification::notify_one;
```

```
inline constexpr auto memory_notify_all = memory_notification::notify_all;
}
```

Adding the following overloads to the primary template for `std::atomic` class:

```
namespace std {
    template<class T> struct atomic {
        // ... existing methods omitted for brevity

        void store(T, memory_notification, memory_order = memory_order::seq_cst)
        void store(T, memory_notification, memory_order = memory_order::seq_cst)

        T exchange(T, memory_notification, memory_order = memory_order::seq_cst)
        T exchange(T, memory_notification, memory_order = memory_order::seq_cst)

        bool compare_exchange_weak(T&, T, memory_notification, memory_order, memc
        bool compare_exchange_weak(T&, T, memory_notification, memory_order, memc
        bool compare_exchange_strong(T&, T, memory_notification, memory_order, me
        bool compare_exchange_strong(T&, T, memory_notification, memory_order, me

        bool compare_exchange_weak(T&, T, memory_notification,
                                   memory_order = memory_order::seq_cst) volatile
        bool compare_exchange_weak(T&, T, memory_notification,
                                   memory_order = memory_order::seq_cst) noexcept
        bool compare_exchange_strong(T&, T, memory_notification,
                                   memory_order = memory_order::seq_cst) volatile
        bool compare_exchange_strong(T&, T, memory_notification,
                                   memory_order = memory_order::seq_cst) noexcept
    };
}
```

Adding the following methods to specialisations of `std::atomic` for integral types

```
namespace std {
    template<> struct atomic<integral> {
        // ... existing methods omitted for brevity

        void store(integral, memory_notification, memory_order = memory_order::se
        void store(integral, memory_notification, memory_order = memory_order::se

        integral exchange(integral, memory_notification, memory_order = memory_or
        integral exchange(integral, memory_notification, memory_order = memory_or

        bool compare_exchange_weak(integral&, integral, memory_notification,
                                   memory_order, memory_order) volatile noexcept;
        bool compare_exchange_weak(integral&, integral, memory_notification,
```



```

        memory_order, memory_order) noexcept;
    bool compare_exchange_strong(integral&, integral, memory_notification,
                                memory_order, memory_order) volatile noexcept;
    bool compare_exchange_strong(integral&, integral, memory_notification,
                                memory_order, memory_order) noexcept;

    bool compare_exchange_weak(integral&, integral, memory_notification,
                               memory_order = memory_order::seq_cst) volatile;
    bool compare_exchange_weak(integral&, integral,
                               memory_notification, memory_order = memory_order::seq_cst) volatile;
    bool compare_exchange_strong(integral&, integral, memory_notification,
                                memory_order = memory_order::seq_cst) volatile;
    bool compare_exchange_strong(integral&, integral, memory_notification,
                                memory_order = memory_order::seq_cst) noexcept;

    integral fetch_add(integral, memory_notification,
                       memory_order = memory_order::seq_cst) const noexcept;
    integral fetch_sub(integral, memory_notification,
                       memory_order = memory_order::seq_cst) const noexcept;
    integral fetch_and(integral, memory_notification,
                       memory_order = memory_order::seq_cst) const noexcept;
    integral fetch_or(integral, memory_notification,
                      memory_order = memory_order::seq_cst) const noexcept;
    integral fetch_xor(integral, memory_notification,
                       memory_order = memory_order::seq_cst) const noexcept;
};

```

Adding similar overloads to `std::atomic_ref` (omitted for brevity)

Adding the following namespace-scope overloads:

```

namespace std {
    template<class T>
        void atomic_store(volatile atomic<T>*, typename atomic<T>::value_type, memory_order) noexcept;
    template<class T>
        void atomic_store(atomic<T>*, typename atomic<T>::value_type, memory_order) noexcept;
    template<class T>
        void atomic_store_explicit(volatile atomic<T>*, typename atomic<T>::value_type, memory_order) noexcept;
    template<class T>
        void atomic_store_explicit(atomic<T>*, typename atomic<T>::value_type, memory_order) noexcept;

    template<class T>
        T atomic_exchange(volatile atomic<T>*, typename atomic<T>::value_type, memory_order) noexcept;
    template<class T>
        T atomic_exchange(atomic<T>*, typename atomic<T>::value_type, memory_order) noexcept;
    template<class T>
        T atomic_exchange_explicit(volatile atomic<T>*, typename atomic<T>::value_type, memory_order) noexcept;
    template<class T>
        T atomic_exchange_explicit(atomic<T>*, typename atomic<T>::value_type, memory_order) noexcept;
}

```

```

    T atomic_exchange_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
                               memory_order) noexcept;

template<class T>
    T atomic_exchange_explicit(atomic<T>*, typename atomic<T>::value_type,
                               memory_order) noexcept;

template<class T>
    bool atomic_compare_exchange_weak(volatile atomic<T>*,
                                       typename atomic<T>::value_type*,
                                       typename atomic<T>::value_type,
                                       memory_order_notification) noexcept;

template<class T>
    bool atomic_compare_exchange_weak(atomic<T>*,
                                       typename atomic<T>::value_type*,
                                       typename atomic<T>::value_type,
                                       memory_order_notification) noexcept;

template<class T>
    bool atomic_compare_exchange_strong(volatile atomic<T>*,
                                       typename atomic<T>::value_type*,
                                       typename atomic<T>::value_type,
                                       memory_order_notification) noexcept;

template<class T>
    bool atomic_compare_exchange_strong(atomic<T>*,
                                       typename atomic<T>::value_type*,
                                       typename atomic<T>::value_type,
                                       memory_order_notification) noexcept;

template<class T>
    bool atomic_compare_exchange_weak_explicit(volatile atomic<T>*,
                                                typename atomic<T>::value_type,
                                                typename atomic<T>::value_type,
                                                memory_order_notification,
                                                memory_order, memory_order) noexcept;

template<class T>
    bool atomic_compare_exchange_weak_explicit(atomic<T>*,
                                                typename atomic<T>::value_type,
                                                typename atomic<T>::value_type,
                                                memory_order_notification,
                                                memory_order, memory_order) noexcept;

template<class T>
    bool atomic_compare_exchange_strong_explicit(volatile atomic<T>*,
                                                  typename atomic<T>::value_type,
                                                  typename atomic<T>::value_type,
                                                  memory_order_notification,
                                                  memory_order, memory_order) noexcept;

template<class T>
    bool atomic_compare_exchange_strong_explicit(atomic<T>*,
                                                  typename atomic<T>::value_type,
                                                  typename atomic<T>::value_type,
                                                  memory_order_notification,
                                                  memory_order, memory_order) noexcept;

```

```

template<class T>
    T atomic_fetch_add(volatile atomic<T>*, typename atomic<T>::difference_ty
template<class T>
    T atomic_fetch_add(atomic<T>*, typename atomic<T>::difference_type, memor
template<class T>
    T atomic_fetch_add_explicit(volatile atomic<T>*, typename atomic<T>::diff
                                memory_order) noexcept;
template<class T>
    T atomic_fetch_add_explicit(atomic<T>*, typename atomic<T>::difference_ty
                                memory_order) noexcept;
template<class T>
    T atomic_fetch_sub(volatile atomic<T>*, typename atomic<T>::difference_ty
template<class T>
    T atomic_fetch_sub(atomic<T>*, typename atomic<T>::difference_type, memor
template<class T>
    T atomic_fetch_sub_explicit(volatile atomic<T>*, typename atomic<T>::diff
                                memory_order) noexcept;
template<class T>
    T atomic_fetch_sub_explicit(atomic<T>*, typename atomic<T>::difference_ty
                                memory_order) noexcept;
template<class T>
    T atomic_fetch_and(volatile atomic<T>*, typename atomic<T>::value_type, n
template<class T>
    T atomic_fetch_and(atomic<T>*, typename atomic<T>::value_type, memory_not
template<class T>
    T atomic_fetch_and_explicit(volatile atomic<T>*, typename atomic<T>::valu
                                memory_order) noexcept;
template<class T>
    T atomic_fetch_and_explicit(atomic<T>*, typename atomic<T>::value_type, n
                                memory_order) noexcept;
template<class T>
    T atomic_fetch_or(volatile atomic<T>*, typename atomic<T>::value_type, me
template<class T>
    T atomic_fetch_or(atomic<T>*, typename atomic<T>::value_type, memory_noti
template<class T>
    T atomic_fetch_or_explicit(volatile atomic<T>*, typename atomic<T>::value
                                memory_order) noexcept;
template<class T>
    T atomic_fetch_or_explicit(atomic<T>*, typename atomic<T>::value_type, me
                                memory_order) noexcept;
template<class T>
    T atomic_fetch_xor(volatile atomic<T>*, typename atomic<T>::value_type, n
template<class T>
    T atomic_fetch_xor(atomic<T>*, typename atomic<T>::value_type, memory_not
template<class T>
    T atomic_fetch_xor_explicit(volatile atomic<T>*, typename atomic<T>::valu
                                memory_order) noexcept;
template<class T>

```

```

    T atomic_fetch_xor_explicit(atomic<T>*, typename atomic<T>::value_type, n
                                memory_order) noexcept;
}

```

Where each of the overloads taking a `memory_notification` is equivalent to executing the corresponding overload without the `memory_notification` parameter, but with the semantic that if the operation stores a value to the atomic variable then it atomically performs the store and a call to the corresponding `notify_one` or `notify_all` methods (depending on the value of the `memory_notification` parameter).

For example, the `store()` overload has the semantics of performing the following body as if performed atomically.

```

template<typename T>
void atomic<T>::store(T value, memory_notification notify, memory_order order)
    // As if it atomically performs the following operations
    store(value, order);
    switch(notify) {
    case memory_notification::notify_one: notify_one(); break;
    case memory_notification::notify_all: notify_all(); break;
    }
}

```

And for the `compare_exchange_strong()` operation, it would atomically perform the notification if the compare-exchange operation succeeds.

```

template<typename T>
bool atomic<T>::compare_exchange_strong(T& old_val, T new_val, memory_notific
                                        memory_order mo_success, memory_order
                                        // As if it atomically performs the following operations
                                        bool result = compare_exchange_strong(old_val, new_val, mo_success, mo_fa
                                        if (result) {
                                            switch (notify) {
                                            case memory_notification::notify_one: notify_one(); break;
                                            case memory_notification::notify_all: notify_all(); break;
                                            }
                                        }
                                        return result;
}

```

In practice, existing implementations should be able to call through to the underlying OS syscall after performing the atomic operation (i.e. call one of `WakeByAddressSingle/All()`

or `futex()` depending on the platform), or on platforms without the OS primitives, in terms of operations on statically-allocated synchronisation state (e.g. using a mutex/condition-variable).

For example: On Windows 8 or later

```
template<>
void atomic<int>::store(int value, memory_notification notify, memory_order c
    void* address = static_cast<void*>(this);
    store(value, order);
    switch (notify) {
    case memory_notification::notify_one: WakeByAddressSingle(address); break;
    case memory_notification::notify_all: WakeByAddressAll(address); break;
    }
}
```

For example: On Linux 2.6.22 or later

```
template<>
void atomic<int>::store(int value, memory_notification notify, memory_order c
    void* address = static_cast<void*>(this);
    store(value, order);
    if (notify != memory_notification_none) {
        (void)syscall(SYS_futex, address, FUTEX_WAKE_PRIVATE,
            notify == memory_notification_one ? 1 : INT_MAX);
    }
}
```

Note that the above implementations still rely on the address being valid to copy and pass as a parameter to the syscall, even after the `store()` has potentially caused the address to become invalid due to the object being destroyed on another thread. However, whether or not this is valid is implementation defined and so presumably implementations that take this approach can ensure this is indeed valid.

For example: On platforms without native OS support for address-based notification

```
struct __wait_state {
    std::atomic<uint64_t> _M_waiters{0};
    std::mutex _M_mut;
    std::condition_variable _M_cv;
    std::uint64_t _M_version{0};

    static __wait_state& __for_address(void* __address) noexcept {
```

```

constexpr std::uintptr_t __count = 16;
static __wait_state __w[__count];
auto __key = (reinterpret_cast<std::uintptr_t>(__address) >> 2) % __c
return __w[__key];
}

void __notify() noexcept {
    if (_M_waiters.load() != 0) {
        {
            std::lock_guard __lk{_M_mut};
            ++_M_version;
        }
        _M_cv.notify_all();
    }
}

template<typename _Pred>
void __wait(_Pred __pred) noexcept {
    for (int __i = 0; __i < 10; ++__i) {
        if (__pred()) return;
        __yield_thread();
    }

    _M_waiters.fetch_add(1, std::memory_order_seq_cst);
    std::uint64_t __prev_version = [&] {
        std::unique_lock __lk{_M_mut};
        return _M_version;
    }();
    while (!__pred()) {
        std::unique_lock __lk{_M_mut};
        if (_M_version == __prev_version) {
            _M_cv.wait(__lk);
        }
        __prev_version = _M_version;
    }
    _M_waiters.fetch_sub(1, std::memory_order_release);
}

template<>
void atomic<int>::store(int __value, memory_notification __notify, memory_order __mo) {
    auto& __w = __wait_state::__for_address(this);
    store(__value, __mo);
    if (__notify != memory_notification_none) {
        __w.__notify();
    }
}

template<>

```

```
void atomic<int>::wait(int __old, memory_order __mo) noexcept {
    auto __pred = [__mo, __old, this]() noexcept { return this->load(__mo) !=
    auto& __w = __wait_state::__for_address(this);
    __w.wait(__pred);
}
```

Other implementations considered

Other variations on option 2 were considered and rejected.

Instead of adding overloads of existing function names, we could have added separately named functions. e.g. `store_and_notify_one` and `store_and_notify_all`.

This was rejected as it adds a large number of new names to the `std::atomic` interface. It also makes it more difficult to conditionally either notify-one or notify-all.

e.g. An implementation of `std::counting_semaphore::release()` can be the following if using the enum-based overload solution:

```
template<std::ptrdiff_t LeastMaxValue>
void std::counting_semaphore<LeastMaxValue>::release(ptrdiff_t update) {
    counter_.fetch_add(update,
                       (update == 1) ? memory_notify_one : memory_notify_all,
                       memory_order_release);
}
```

whereas with separate names you would need to write:

```
template<std::ptrdiff_t LeastMaxValue>
void std::counting_semaphore<LeastMaxValue>::release(ptrdiff_t update) {
    if (update == 1) {
        counter_.fetch_add_and_notify_one(update, memory_order_release);
    } else {
        counter_.fetch_add_and_notify_all(update, memory_order_release);
    }
}
```

Limitations

This option does have some limitations, which were pointed out on the reflector.

Consider the use-cases where the semaphore implementation above wants to conditionally notify other threads only if we are incrementing the available count from zero (acquiring threads call `counter_.wait(0)`).

```
template<std::ptrdiff_t LeastMaxValue>
void std::counting_semaphore<LeastMaxValue>::release(ptrdiff_t update) {
    if (counter_.fetch_add(update, memory_order_release) == 0)
        counter_.notify_all();
}
```

The fused write+notify operations do not provide enough flexibility to support this conditional notification safely without resorting to a compare-exchange loop.

For example:

```
template<std::ptrdiff_t LeastMaxValue>
void std::counting_semaphore<LeastMaxValue>::release(ptrdiff_t update) {
    auto old = counter_.load(memory_order_relaxed);
    while (!counter_.compare_exchange_weak(old, old + 1,
                                           old == 0 ? memory_order_notify_all : memory_order_release,
                                           memory_order_relaxed, memory_order_relaxed))
        {}
}
```

Option 3 - Atomic Notify Token

The final option considered here takes the approach of adding a method that lets users obtain from an atomic a token that can later be used to notify entities waiting on the atomic from which it was obtained, even if the atomic object has since been destroyed.

Synopsis:

```
namespace std {
    template<typename T>
    class atomic_notify_token;

    template<typename T>
    class atomic {
    public:
        // Existing members...

        atomic_notify_token<T> get_notify_token() noexcept;
```



```

};

template<typename T>
class atomic_ref {
public:
    // Existing members...

    atomic_notify_token<T> get_notify_token() noexcept;
};

template<typename T>
class atomic_notify_token {
public:
    // Copyable
    atomic_notify_token(const atomic_notify_token&) noexcept = default;
    atomic_notify_token& operator=(const atomic_notify_token&) noexcept = default;

    // Perform notifications
    void notify_one() const noexcept;
    void notify_all() const noexcept;
private:
    // exposition-only
    friend class atomic<T>;
    explicit atomic_notify_token(std::uintptr_t p) noexcept : address(p) {}
    std::uintptr_t address;
};
}

```

The idea behind this is that the token can hold enough information to be able to notify waiting threads without necessarily doing anything that might cause undefined behaviour if the atomic object is potentially destroyed.

Potential Implementation Strategies

On platforms that use a `futex()` -like OS API to implement waiting, the token could hold the address of the object (e.g. as a `void*` or `std::uintptr_t`) assuming on this particular implementation that was valid if the object is subsequently destroyed.

Example: Possible implementation on Windows

```

template<typename T>
class atomic_notify_token {
public:
    void notify_one() noexcept { WakeByAddressSingle(ptr); }

```

```

void notify_all() noexcept { WakeByAddressAll(ptr); }
private:
    friend class atomic<T>;
    explicit atomic_notify_token(void* p) noexcept : ptr(p) {}
    void* ptr;
};

```

On platforms without native OS support for address-based notification, the token could hold a pointer to the state used to manage the waiters.

For example: Using the `__wait_state*` from the example described in Option 2 above.

```

template<typename T>
class atomic_notify_token {
public:
    void notify_one() noexcept { __state->__notify(); }
    void notify_all() noexcept { __state->__notify(); }
private:
    explicit atomic_notify_token(__wait_state* __s) noexcept : __state(__s) {}
    __wait_state* __state_;
};

template<typename T>
atomic_notify_token<T> atomic<T>::get_notify_token() noexcept {
    __wait_state& __s = __wait_state::__for_address(this);
    return atomic_notify_token<T>{&__s};
}

```

In this case, the `__wait_state*` object is computed from the address of the atomic object while the atomic object is still alive. Once the address of the `__wait_state` object is computed there is no need for the atomic object address any more - and thus no potential for using a pointer to a potentially destroyed object.

Why is `atomic_notify_token<T>` a template?

The token type is a class-template to allow for specialisations of `atomic_notify_token<T>` to use different strategies for different `T` types.

On some platforms, the native os wait/notify functions only work with certain sized types. e.g. 4 byte integers. Types with a different size might dispatch to a different implementation that uses a proxy 4-byte value for waiting.

Example usage

The `counting_semaphore::release()` method could be implemented as follows:

```
template<std::ptrdiff_t LeastMaxValue>
void std::counting_semaphore<LeastMaxValue>::release(ptrdiff_t update) {
    // Obtain the token before the operation that might end the lifetime
    auto token = counter_.get_notify_token();
    if (counter_.fetch_add(update, memory_order_release) == 0) {
        // Use the token after the operation that might end the lifetime to notify
        token.notify_all();
    }
}
```

Conclusion

The current separation of `std::atomic::notify_one/all` from the store operation makes it difficult/impossible to use efficiently and safely for scenarios where a preceding write to the atomic object may cause the atomic object to be destroyed.

This limits the ability for users to implement synchronisation primitives like `std::counting_semaphore` in terms of `std::atomic` without relying on implementation-specific behaviour.

This paper lists three potential design directions we could take to address this shortcoming:

1. We can pursue language improvements that make it valid to call (some of) the existing notifying functions with the address of a potentially destroyed atomic object.
2. We can pursue a library solution that provides new overloads to atomic operations that as-if atomically store a value and notify waiting threads so that we avoid the lifetime issues.
3. We can pursue a library solution based on obtaining a token that can be used to notify waiting threads safely, even if the original atomic object is potentially destroyed.

I am seeking guidance from the Concurrency sub-group on the preferred direction and can produce proposed wording in a subsequent revision based on this guidance.

References

SG1 mailing list thread on the issue <https://lists.isocpp.org/parallel/2020/07/3270.php>

P1726R5 - "Pointer lifetime-end zap and provenance, too" (Paul McKenney, Maged Michael, et. al.) <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1726r5.pdf>

P2188R1 - "Zap the Zap: Pointers are sometimes just bags of bits" <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2188r1.html>